ADESH PATANKAR

BDT LAB: 10

Problem Statement: Writing a program in Spark for Word Count with Unit Tests

Outcome: Students will be able to implement application for counting number of words in a text file using Apache Spark

Theory:

Apache Spark has emerged as a leading distributed computing framework for processing large-scale data efficiently and reliably. It offers a unified platform for various data processing tasks, including batch processing, stream processing, machine learning, and interactive queries. Spark's popularity stems from its ease of use, scalability, and rich set of APIs, which enable developers to build complex data pipelines with ease.

Spark's core abstraction, the Resilient Distributed Dataset (RDD), lies at the heart of its computation model. RDDs provide fault tolerance and parallelism, allowing data to be distributed across a cluster of machines and processed in parallel. This enables Spark to handle massive datasets that exceed the capacity of a single machine, making it suitable for processing big data.

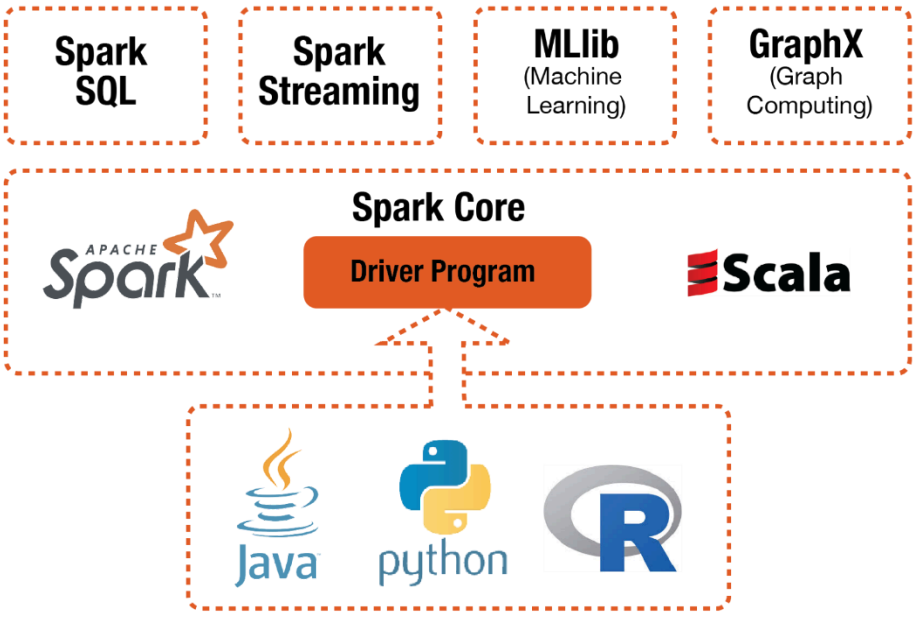


Fig. 1. Apache Spark ecosystem

The Spark ecosystem includes libraries and tools for diverse use cases:

Spark SQL: Provides a DataFrame API for working with structured data, enabling SQL queries and integration with traditional databases.

Spark Streaming: Enables real-time processing of streaming data, allowing for the analysis of data as it arrives.

MLlib: Offers scalable machine learning algorithms and tools for building predictive models on large datasets.

GraphX: Facilitates graph processing and analysis, allowing for the computation of graph algorithms on large-scale graphs.

With its distributed computing capabilities, Spark can run on clusters ranging from a single machine to thousands of nodes, making it suitable for a wide range of use cases. Whether processing terabytes of log data, analyzing streaming data in real-time, or training machine learning models on massive datasets, Apache Spark provides a versatile platform for big data analytics and processing.

In this context, the Word Count example serves as a foundational illustration of Spark's capabilities. By implementing Word Count, developers can gain insights into Spark's core concepts, including RDDs, transformations, actions, and parallel processing. Additionally, Word Count provides a practical demonstration of how Spark can handle common data processing tasks efficiently and scale seamlessly to meet the demands of large-scale data processing application.

Resilient Distributed Dataset (RDD):

RDD is the fundamental data structure of Apache Spark, representing an immutable distributed collection of objects that can be operated on in parallel. RDDs are fault-tolerant, meaning they can automatically recover from failures by using lineage information to recompute lost data partitions. They support two types of operations: transformations and actions.

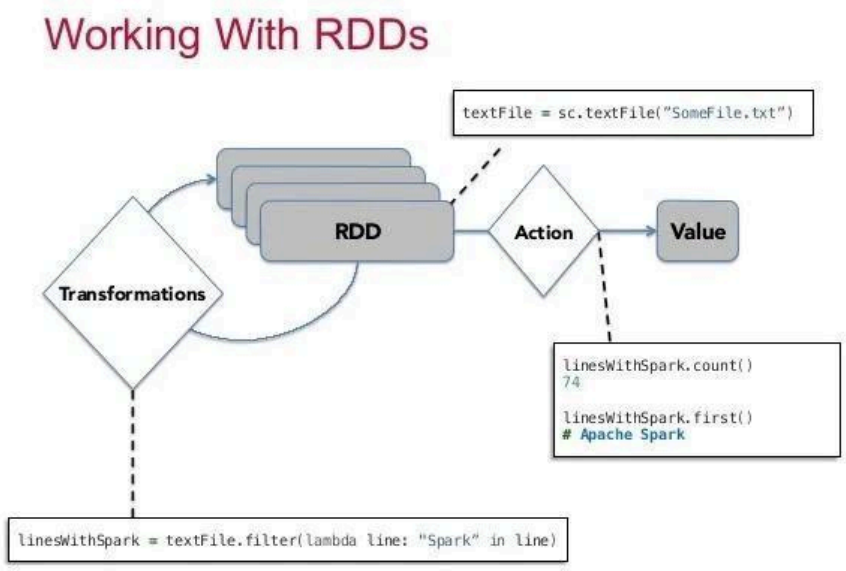


Fig. 2. Working with RDD

Transformations:

Transformations create a new dataset from an existing one by applying a function. Examples include `map`, `flatMap`, `filter`, `reduceByKey`, etc. Transformations are lazily evaluated, meaning they are not executed until an action is triggered.

Actions:

Actions return a value to the driver program after running a computation on the dataset. Examples include `collect`, `count`, `saveAsTextFile`, `reduce`, etc. Actions trigger the execution of the underlying Spark computation and may trigger stages of execution.

Word Count:

Word Count is a simple application used to count the frequency of each word in a text document. It's a classic example for learning distributed data processing frameworks like Spark due to its simplicity and effectiveness. The input is typically a text file, and the output is a set of word-count pairs. The

algorithm involves loading the text file, splitting it into words, counting the occurrences of each word, and then aggregating the results.

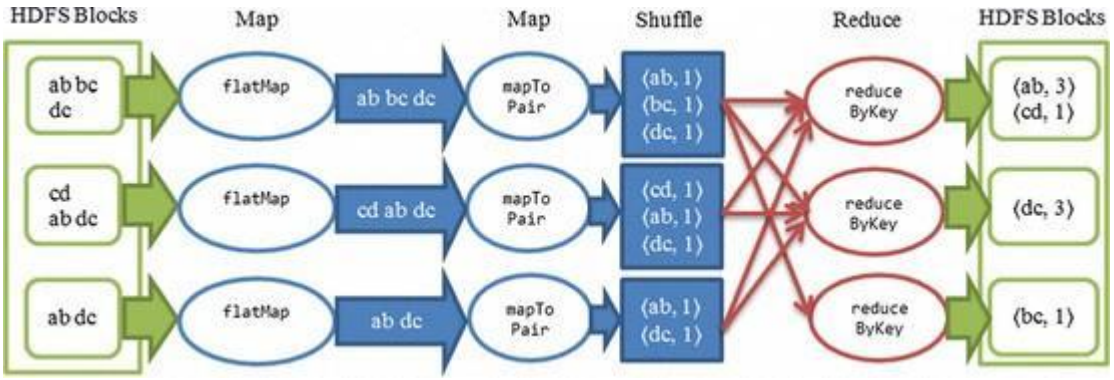


Fig. 2. Word count example in Apache Spark framework

Implementation Steps:

Create Spark Context: Initialize a SparkConf object with the application name and create a SparkContext.

Load Data: Read the input text file into an RDD using the `textFile` function.

Split Words: Use the `flatMap` transformation to split each line into words.

Map to Key-Value Pairs: Transform each word into a key-value pair `(word, 1)`.

Reduce by Key: Aggregate the word counts using the `reduceByKey` transformation.

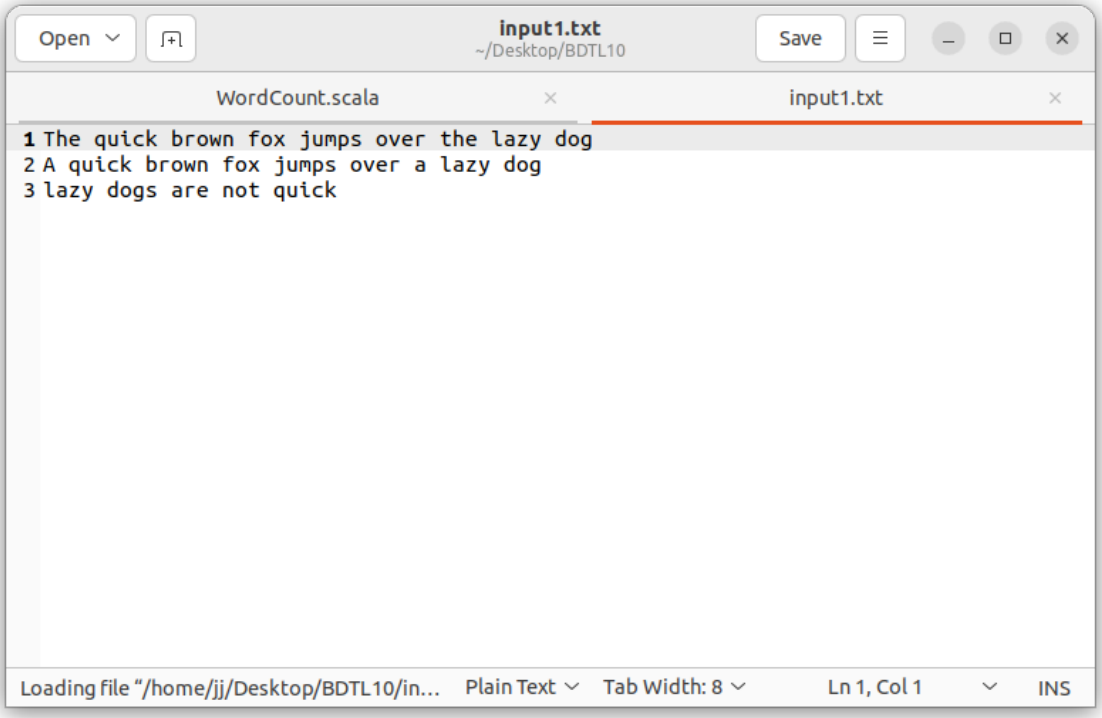
Save Output: Save the word counts to a text file using the `saveAsTextFile` action.

Stop Spark Context: Terminate the SparkContext to release resources and shut down the Spark application.

Word Count is a foundational example in distributed data processing and serves as an excellent starting point for learning Apache Spark. By understanding the principles of RDDs, transformations, and actions, one can implement the Word Count application efficiently, demonstrating the distributed computing capabilities of Spark.

Results:

Input file:



WordCount.scala file:



Starting the Apache Spark CLI:

```
jj@jj-VirtualBox:~$ spark-shell
24/04/23 14:32:16 WARN Utils: Your hostname, jj-VirtualBox resolves to a loopback a
ddress: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
24/04/23 14:32:16 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another add
ress
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(n
ewLevel).
24/04/23 14:32:21 WARN NativeCodeLoader: Unable to load native-hadoop library for y
our platform... using builtin-java classes where applicable
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1713862941998).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.4.3
      /_/

Using Scala version 2.12.17 (OpenJDK 64-Bit Server VM, Java 1.8.0_402)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

1. **Importing the necessary Spark classes**

```
scala> import org.apache.spark.{SparkConf, SparkContext}
import org.apache.spark.{SparkConf, SparkContext}
```

2. **Create Spark configuration**

```
scala> val conf = new SparkConf().setAppName("WordCount")
conf: org.apache.spark.SparkConf = org.apache.spark.SparkConf@5b41f249
```

3. **Create Spark context**

```
scala> val sc = new SparkContext(conf)
sc: org.apache.spark.SparkContext = org.apache.spark.SparkContext@32fd55a6
```

4. **Load input data**

```
scala> val input = sc.textFile("hdfs:///Lab/input1.txt")
input: org.apache.spark.rdd.RDD[String] = hdfs:///Lab/input1.txt MapPartitionsRDD[1
] at textFile at <console>:26
```

5. **Split each line into words**

```
scala> val words = input.flatMap(line => line.split(" "))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at flatMap at <consol
e>:26
```

6. **Count the occurrence of each word**

```
scala> val wordCounts = words.map(word => (word, 1)).reduceByKey(_ + _)
```

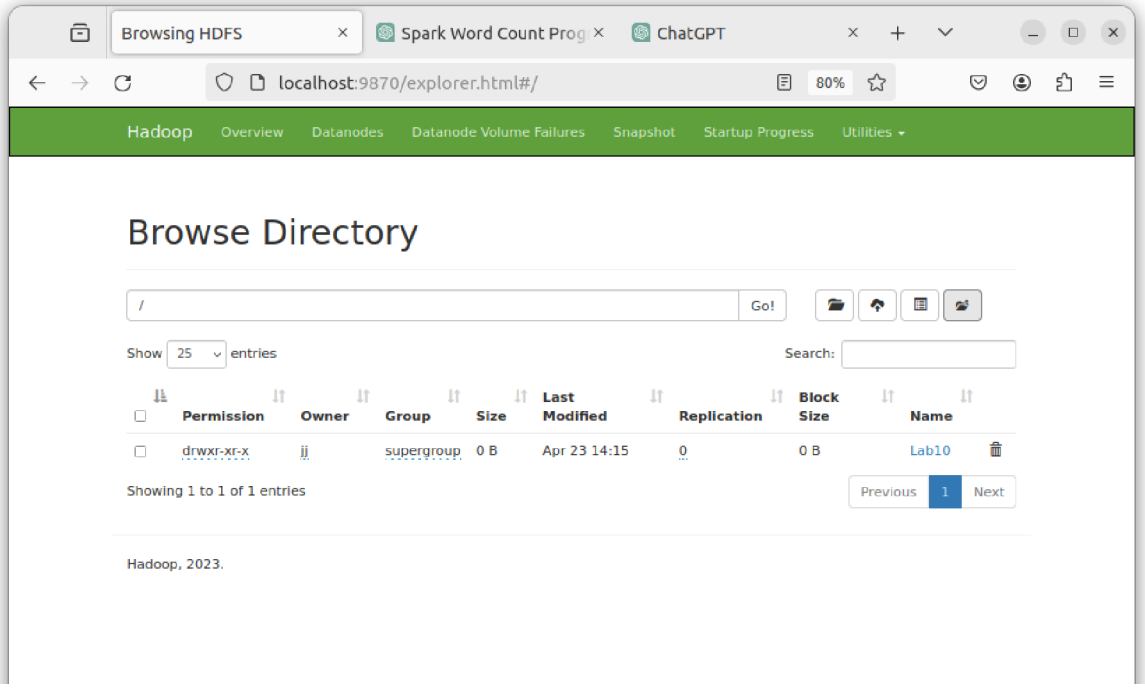
7. **Save the word count result to a file**

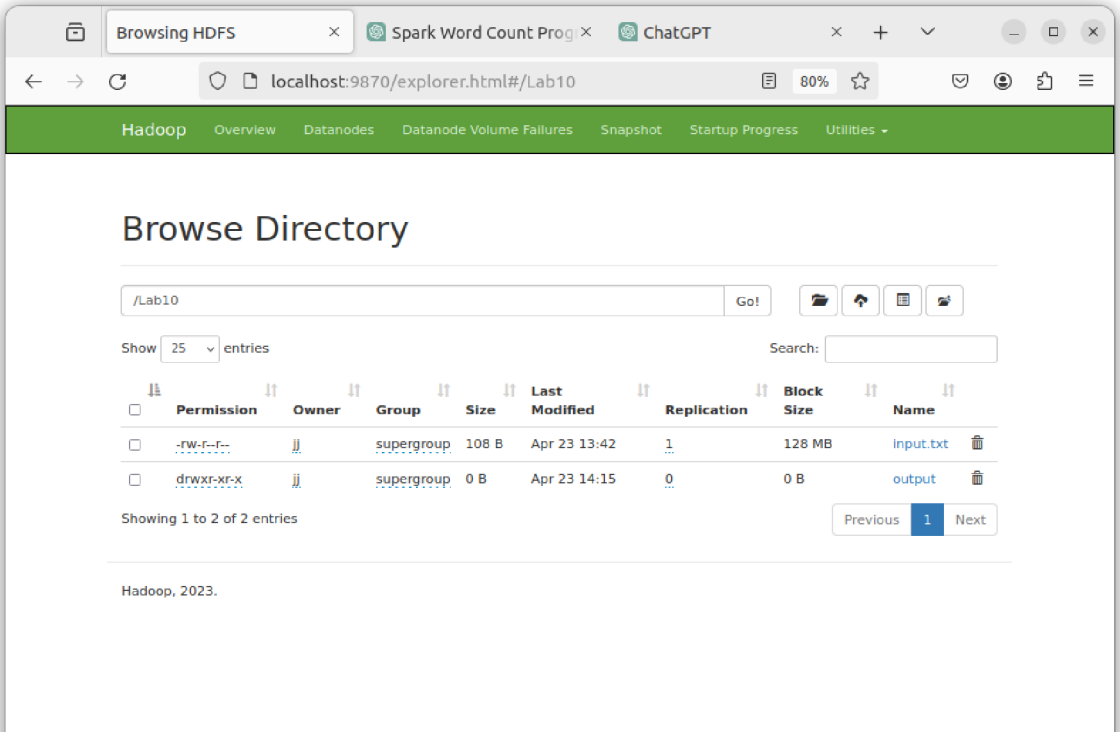
```
scala> wordCounts.saveAsTextFile("hdfs:///Lab10/output")
```

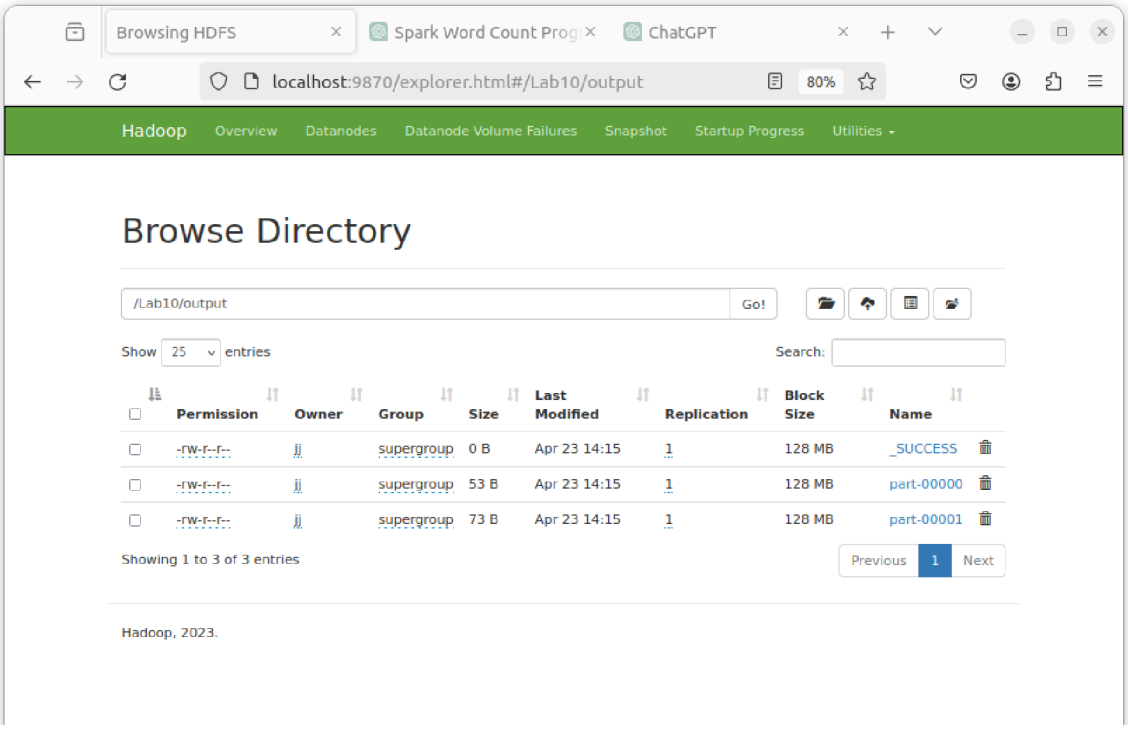
8. **Stop Spark context**

```
scala> sc.stop()
```

Hadoop output screen:



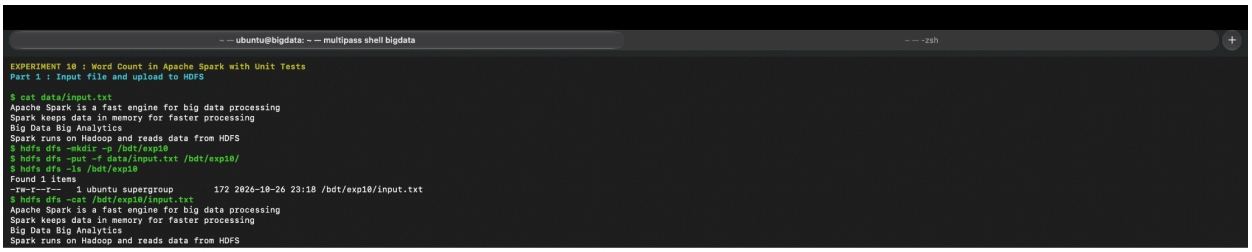

Browsing HDFS
Spark Word Count Prog
ChatGPT
localhost:9870/explorer.html#/Lab10
80%
Hadoop
Overview
Datanodes
Datanode Volume Failures
Snapshot
Startup Progress
Utilities
Browse Directory
/Lab10
Go!
Show 25 entries
Search:
Permission
Owner
Group
Size
Last Modified
Replication
Block Size
Name
-rw-r--r--
jj
supergroup
108 B
Apr 23 13:42
1
128 MB
input.txt
drwxr-xr-x
jj
supergroup
0 B
Apr 23 14:15
0
0 B
output
Showing 1 to 2 of 2 entries
Previous
1
Next
Hadoop, 2023.


Browsing HDFS
Spark Word Count Prog
ChatGPT
localhost:9870/explorer.html#/Lab10/output
80%
Hadoop
Overview
Datanodes
Datanode Volume Failures
Snapshot
Startup Progress
Utilities
Browse Directory
/Lab10/output
Go!
Show 25 entries
Search:
Permission
Owner
Group
Size
Last Modified
Replication
Block Size
Name
-rw-r--r--
jj
supergroup
0 B
Apr 23 14:15
1
128 MB
_SUCCESS
-rw-r--r--
jj
supergroup
53 B
Apr 23 14:15
1
128 MB
part-00000
-rw-r--r--
jj
supergroup
73 B
Apr 23 14:15
1
128 MB
part-00001
Showing 1 to 3 of 3 entries
Previous
1
Next
Hadoop, 2023.

Output:

```
jj@jj-VirtualBox: ~
jj@jj-VirtualBox:~$ hadoop fs -cat hdfs:///Lab10/output/*
(are,1)
(lazy,2)
(Lazy,1)
(dog,2)
(over,2)
(brown,2)
(fox,2)
(not,1)
(The,1)
(jumps,2)
(a,1)
(quick,3)
(A,1)
(dogs,1)
(the,1)
jj@jj-VirtualBox:~$
```

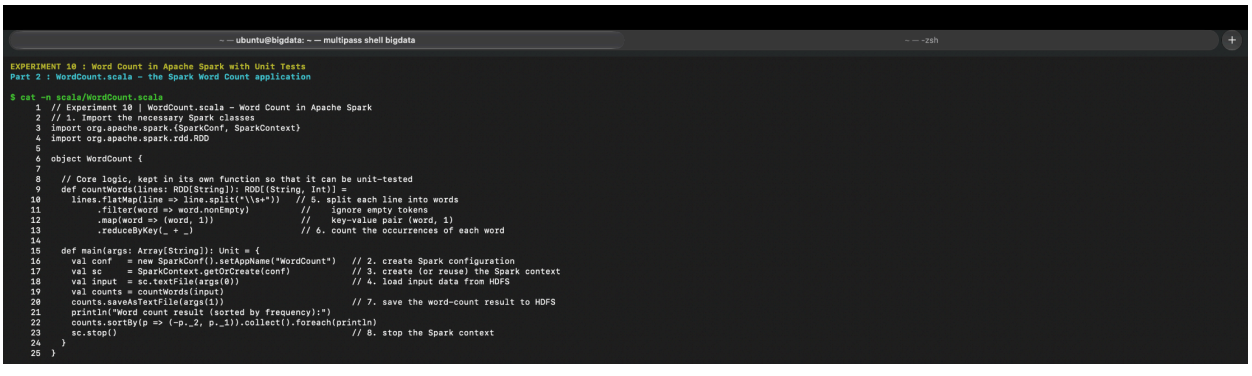
ScreenShots :



```
~ — ubuntu@bigdata: ~ — multipass shell bigdata
~ — -zsh

EXPERIMENT 10 : Word Count in Apache Spark with Unit Tests
Part 1 : Input file and upload to HDFS

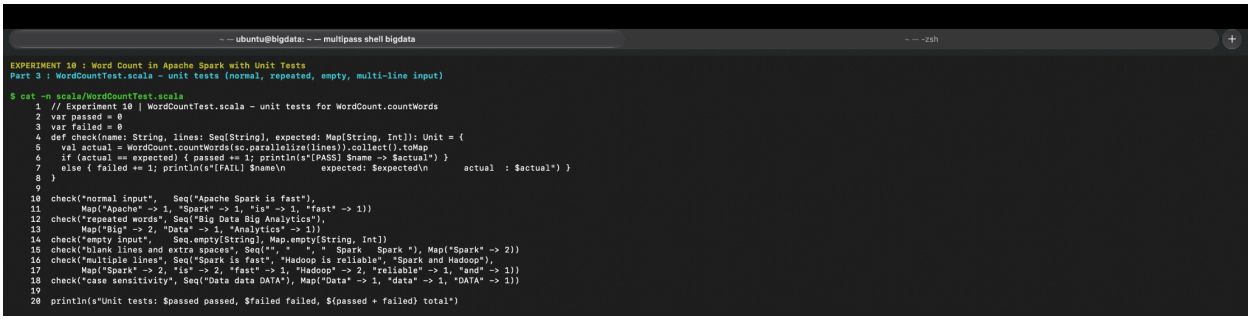
$ cat data/input.txt
Apache Spark is a fast engine for big data processing
Spark keeps data in memory for faster processing
Big Data Big Analytics
Spark runs on Hadoop and reads data from HDFS
$ hdfs dfs -mkdir -p /bdt/exp10
$ hdfs dfs -put -f data/input.txt /bdt/exp10/
$ hdfs dfs -ls /bdt/exp10
Found 1 items
-rw-r--r--   1 ubuntu supergroup        172 2026-10-26 23:18 /bdt/exp10/input.txt
$ hdfs dfs -cat /bdt/exp10/input.txt
Apache Spark is a fast engine for big data processing
Spark keeps data in memory for faster processing
Big Data Big Analytics
Spark runs on Hadoop and reads data from HDFS
```



```
~ — ubuntu@bigdata: ~ — multipass shell bigdata
~ — -zsh

EXPERIMENT 10 : Word Count in Apache Spark with Unit Tests
Part 2 : WordCount.scala - the Spark Word Count application

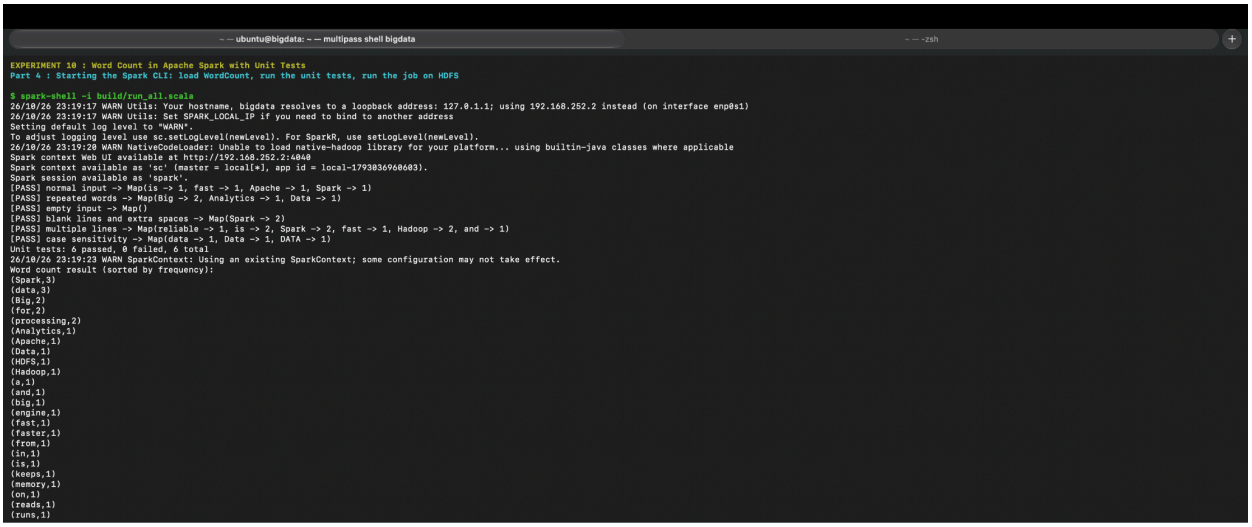
$ cat -n scala/WordCount.scala
     1  // Experiment 10 | WordCount.scala - Word Count in Apache Spark
     2  // 1. Import the necessary Spark classes
     3  import org.apache.spark.{SparkConf, SparkContext}
     4  import org.apache.spark.rdd.RDD
     5
     6  object WordCount {
     7
     8    // Core logic, kept in its own function so that it can be unit-tested
     9    def countWords(lines: RDD[String]): RDD[(String, Int)] =
    10      lines.flatMap(line => line.split("\\s+"))   // 5. split each line into words
    11           .filter(word => word.nonEmpty)         //    ignore empty tokens
    12           .map(word => (word, 1))                //    key-value pair (word, 1)
    13           .reduceByKey(_ + _)                    // 6. count the occurrences of each word
    14
    15    def main(args: Array[String]): Unit = {
    16      val conf   = new SparkConf().setAppName("WordCount")   // 2. create Spark configuration
    17      val sc     = SparkContext.getOrCreate(conf)            // 3. create (or reuse) the Spark context
    18      val input  = sc.textFile(args(0))                      // 4. load input data from HDFS
    19      val counts = countWords(input)
    20      counts.saveAsTextFile(args(1))                         // 7. save the word-count result to HDFS
    21      println("Word count result (sorted by frequency):")
    22      counts.sortBy(p => (-p._2, p._1)).collect().foreach(println)
    23      sc.stop()                                              // 8. stop the Spark context
    24    }
    25  }
```

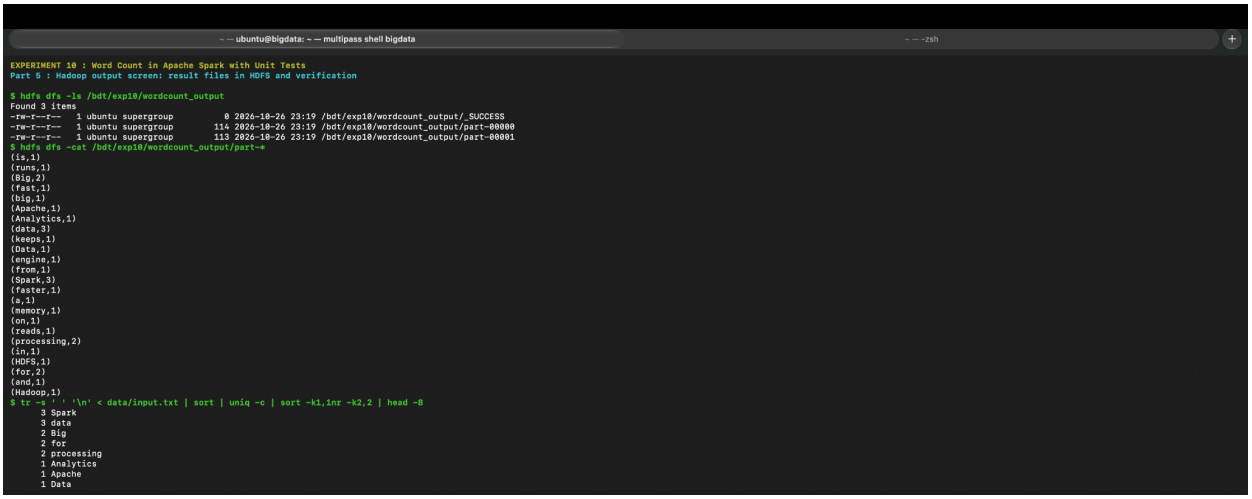


```
~ — ubuntu@bigdata: ~ — multipass shell bigdata
~ — -zsh

EXPERIMENT 10 : Word Count in Apache Spark with Unit Tests
Part 3 : WordCountTest.scala - unit tests (normal, repeated, empty, multi-line input)

$ cat -n scala/WordCountTest.scala
     1  // Experiment 10 | WordCountTest.scala - unit tests for WordCount.countWords
     2  var passed = 0
     3  var failed = 0
     4  def check(name: String, lines: Seq[String], expected: Map[String, Int]): Unit = {
     5    val actual = WordCount.countWords(sc.parallelize(lines)).collect().toMap
     6    if (actual == expected) { passed += 1; println(s"[PASS] $name -> $actual") }
     7    else { failed += 1; println(s"[FAIL] $name\n       expected: $expected\n       actual  : $actual") }
     8  }
     9
    10  check("normal input",   Seq("Apache Spark is fast"),
    11        Map("Apache" -> 1, "Spark" -> 1, "is" -> 1, "fast" -> 1))
    12  check("repeated words", Seq("Big Data Big Analytics"),
    13        Map("Big" -> 2, "Data" -> 1, "Analytics" -> 1))
    14  check("empty input",    Seq.empty[String], Map.empty[String, Int])
    15  check("blank lines and extra spaces", Seq("", "   ", "  Spark   Spark "), Map("Spark" -> 2))
    16  check("multiple lines", Seq("Spark is fast", "Hadoop is reliable", "Spark and Hadoop"),
    17        Map("Spark" -> 2, "is" -> 2, "fast" -> 1, "Hadoop" -> 2, "reliable" -> 1, "and" -> 1))
    18  check("case sensitivity", Seq("Data data DATA"), Map("Data" -> 1, "data" -> 1, "DATA" -> 1))
    19
    20  println(s"Unit tests: $passed passed, $failed failed, ${passed + failed} total")
```





Handwritten Questions

1. Explain the architecture and working of a Spark-based Word Count application. Describe the roles of transformations and actions in its implementation.
2. Write a Spark program to read a text file, split the text into words, and calculate the frequency of each distinct word.
3. Consider the input text Big Data Big Analytics. Write the expected word-count output and explain how the result can be verified against the input.
4. What is unit testing? Write test cases for a Spark Word Count application to verify normal input, repeated words, empty input, and input containing multiple lines.

Q1)

Apache spark is open source distributed computing framework used to process large amount of data quickly

Key features

- fast in-memory data processing
- Support batch and real time processing
- Provides libraries for SQL, ML, graph processing.
- support multiple programming language.

Q2)

Steps

1) Intall JAVA
2) Download and extract Apache Spark
3) Set the environmental variable.
4) Add spark bin directory to system path
5) Verify installing using spark-shell.

Q. 3)

→ 1) JAVA – version - checks the installed java version.

2) echo $JAVA_HOME - JAVA istallation path on Linux

3) echo $Spark_Home - Spark installation path on Linux

4) spark-shell - Launches the interactive spark-shell

Q 4)

→

```
from pyspark.SQL import SparkSession

spark = SparkSession.builder.name("Sum").
        getorcreate()

number = spark.sparkcontext.parallelize
         ([1, 2, 3, 4, 5])
result = number.Sum()

print("Sum=", result)
spark.stop()
```

Acceptance Criteria

1	The student develops a Spark Word Count application that successfully reads an input text file and processes its contents.
2	The application correctly tokenizes the input and calculates the frequency of each distinct word.
3	The student implements and executes unit tests covering normal input, repeated words, empty input, and multiple-line input, as applicable to the implementation.
4	The application passes the defined test cases, produces the expected results, and demonstrates correctness and reliability through test evidence.

Conclusion: A Word Count program was written in Spark (Scala). It read a text file from HDFS, split it into words using flatMap, counted each word using map and reduceByKey, and saved the result back to HDFS. All 6 unit tests passed: normal input, repeated words, empty input, blank lines, multiple lines and case sensitivity. The output matched a manual count done with Linux commands. This shows how transformations and actions in Spark process data in parallel.